Day 1 – Angular Environment & Workspace Setup

🏷️ Setup & Prerequisites

1. **Setup Checklist**:
   - **Install Node.js (LTS)**: Ensure you have Node.js installed (LTS version is recommended). You can verify this by running `node -v` in your terminal.
   - **Install npm**: npm is installed automatically with Node.js. Check the version with `npm -v`.
   - **Open a Clean Terminal**: Open your terminal application and navigate to your active development workspace directory.
   - **Web Browser Ready**: Have a web browser ready to navigate to `http://localhost:4200/`.
2. **Prerequisites**:
   - A working terminal console for running Angular CLI commands.
   - A code editor (like VS Code) with Angular Language Service extension installed.

🎯 Learning Objectives

By the end of this class, you will be able to:

- Install the Angular CLI globally using `npm`.
- Create and scaffold a new Angular workspace using `ng new` with Angular 22 LTS defaults.
- Run the Angular development server using `ng serve` and verify the home page.
- Explain the purpose of key configuration files (`angular.json`, `package.json`, `tsconfig.json`).
- Configure a Git repository and prevent dependencies leak using `.gitignore`.

⏱️ Classroom Timeline (90 min)

Segment	Duration	Focus
1. Hook & Big Picture	15 min	Overview of Single Page Application (SPA) architecture and Angular's role in the frontend.

2. Key Concepts & Core Ideas	15 min	Understanding Angular Workspace structure, CLI toolchain, and configuration files.
3. Live Coding Walkthrough	45 min	Installing the CLI, scaffolding the project, configuring routing, and running the dev server.
4. Check for Understanding	10 min	Q&A on Angular compilation, SPA routing, and node_modules management.
5. Class Wrap-up & Checkpoint	5 min	Verification of local developer environments.

🚀 Step-by-Step Walkthrough

1. Hook & Big Picture (15 min)

In modern web development, we separate the backend database/API logic from the user interface. Angular is a frontend web framework that runs in the user's browser as a Single Page Application (SPA). Instead of request-response cycles rendering new HTML pages from the server on every click, Angular intercepts routing, loads once, and dynamically rewrites parts of the page for instant, fluid interactions.

- **Big Picture**: The client-side flow: Browser requests page → Server returns index.html and compiled JS bundles → Angular bootstraps and renders components → Angular fetches database details dynamically from our Orders/Games API using JSON.
- **Q&A**:
  - **Question**: Why do we build a Single Page Application (SPA) instead of server-side rendered pages?
    - *Answer*: SPAs provide a desktop-like user experience. Since the browser only loads the core Javascript bundle once, subsequent navigation is instantaneous because we do not fetch new HTML pages from the server; we only fetch raw data (JSON) as needed.

2. Key Concepts & Core Ideas (15 min)

Introduce these core Angular concepts:

- **Angular CLI (Command Line Interface)**: A command-line tool used to initialize, develop, scaffold, test, and maintain Angular applications.
- **Angular Workspace**: The root folder containing configuration files and one or more Angular projects (usually a single application).
- **Standalone Components**: Starting in modern Angular, modules (`NgModule`) are optional. Components are self-contained and explicitly declare their own imports, simplifying scaffolding.
- **Webpack / Vite Bundler**: Under the hood, Angular uses Vite and Esbuild to bundle all TypeScript, CSS, and asset files into single, optimized Javascript bundles for browser execution.

- **`node_modules`**: The directory where npm installs all third-party libraries. This folder must NEVER be committed to Git due to its massive size.

---

3. Live Coding Walkthrough (45 min)

Step 3.1: Angular CLI Installation & Project Scaffolding

- **Concept**: Install the Angular CLI globally on your system, then create a new project workspace. We will configure routing and select standard CSS for styling.
- **Code**:

```
# Install the Angular CLI globally on your system
npm install -g @angular/cli

# Scaffold a new Angular application named 'gamekey-store'
# Flags explained:
# --routing=true: Automatically configure Angular Router routing modules.
# --style=css: Set the CSS preprocessor to standard Vanilla CSS.
# --ssr=false: Disable Server-Side Rendering (we are building a pure client-side SPA).
ng new gamekey-store --routing=true --style=css --ssr=false

# Move into the newly created project folder
cd gamekey-store
```

- **Verify**: Check that the project folder contains folders like `src`, `public`, and configuration files like `angular.json` and `package.json`.

Step 3.2: Exploring Workspace Configuration Files

- **Concept**: Understand the files created by the scaffolding command.
- **Files**:
  - `package.json`: Lists the project dependencies (Angular framework, RxJS, etc.) and npm scripts.
  - `angular.json`: Core Angular CLI build configuration file defining build targets, styles, and assets.
  - `tsconfig.json`: TypeScript compiler configurations.
  - `src/main.ts`: The entry point that boots (bootstraps) the Angular application in the browser.
  - `src/index.html`: The single shell HTML file where the Angular application renders.
  - `src/app/app.component.ts`: The root component of the application.

Step 3.3: Running the Development Server

- **Concept**: Start the local development server to compile the application and watch for source code changes.
- **Code**:

```
# Start the Angular development server
ng serve
```

- **Verify**: Open your browser and navigate to `http://localhost:4200/`. Confirm that the Angular welcome screen renders.
- ⚠ **Common Pitfalls**:
  - ○ **Port Collision**: If port `4200` is already in use by another running app, the CLI will ask if you want to run on a different port. Press `Y` to let it assign a new port (e.g., `4201`).
  - ○ **Missing Dependencies**: If you copy the project to another machine without copying `node_modules`, you must run `npm install` before running `ng serve`.

Step 3.4: Git Version Control Initialization

- **Concept**: Setup version control. Verify that the `.gitignore` file correctly ignores node_modules and build output folders to keep repository sizes small.
- **Code**:

```
# Initialize git repository
git init
```

Ensure the auto-generated `.gitignore` file contains:

```
/node_modules/
/dist/
/.angular/
.DS_Store
```

- **Verify**: Run `git status` and confirm that `node_modules` or `.angular` folders are not listed in untracked files.

---

4. Check for Understanding (10 min)

> ***Question**: "What is the purpose of the `src/main.ts` file in an Angular application?"*
> ***Answer**: `main.ts` is the bootstrap script. It is the first file loaded by the browser. Its job is to initialize the Angular environment and boot (bootstrap) the root component (usually `AppComponent`) to render the interface inside `index.html`.*

> ***Question**: "Why do we see `<app-root></app-root>` in `index.html`? What replaces it?"*
> ***Answer**: `<app-root>` is a custom HTML tag called a component selector. When Angular boots up, it looks for the component matching selector `app-root` (defined in `app.component.ts`), compiles its*

> *template HTML, and replaces the `<app-root>` tags in `index.html` with the compiled template.*

> ***Question**: "What's the difference between `dependencies` and `devDependencies` in `package.json`?"*
> ***Answer**: `dependencies` are libraries required to run the application in production (e.g., Angular core libraries, RxJS). `devDependencies` are only needed during local development and build compilation (e.g., Angular CLI compiler, TypeScript, testing frameworks).*

---

5. Daily Checkpoint & Wrap-up (5 min)

- **Verification Criteria**:
  - ○ Running `ng serve` boots up without compilation errors.
  - ○ The browser page at `http://localhost:4200/` loads successfully.
  - ○ Running `git status` shows no untracked `node_modules` or `dist` files.